blocks of locks and condition variables and revisit the design of those basic data structures before moving on to more complex data structures. In chapter 7 we'll look at how to go right back to basics and use the atomic operations described in chapter 5 to build data structures without locks.

So, without further ado, let's look at what's involved in designing a data structure for concurrency.

6.1. What does it mean to design for concurrency?

At the basic level, designing a data structure for concurrency means that multiple threads can access the data structure concurrently, either performing the same or distinct operations, and each thread will see a self-consistent view of the data structure. No data will be lost or corrupted, all invariants will be upheld, and there'll be no problematic race conditions. Such a data structure is said to be *thread-safe*. In general, a data structure will be safe only for particular types of concurrent access. It may be possible to have multiple threads performing one type of operation on the data structure concurrently, whereas another operation requires exclusive access by a single thread. Alternatively, it may be safe for multiple threads to access a data structure concurrently if they're performing *different* actions, whereas multiple threads performing the *same* action would be problematic.

Truly designing for concurrency means more than that, though: it means providing the *opportunity for concurrency* to threads accessing the data structure. By its very nature, a mutex provides *mutual exclusion*: only one thread can acquire a lock on the mutex at a time. A mutex protects a data structure by explicitly *preventing* true concurrent access to the data it protects.

This is called *serialization*: threads take turns accessing the data protected by the mutex; they must access it serially rather than concurrently. Consequently, you must put careful thought into the design of the data structure to enable true concurrent access. Some data structures have more scope for true concurrency than others, but in all cases the idea is the same: